

编程实战项目合集

AI 编程实验室 · 精选教程

¥29.9

生成日期: 2026年05月06日

32 篇精选文章

目录

1. 目录

2. 1. 为什么需要 Docker?

3. 2. 准备工作：安装 Docker

4. 3. 编写一个极简的 Node.js 应用

5. 4. 编写 Dockerfile：描述你的容器

6. 5. 构建并运行你的第一个容器

7. 6. 实际场景：为什么这样写才是最佳实践?

8. 总结

9. 1. 理解异步编程的核心：协程与事件循环

10. 2. 实际场景：异步HTTP请求

11. 3. 控制并发：限制同时进行的任务数

12. 4. 超💎💎💎处理与错误恢复

13. 5. 异步与同步代码的桥接

14. 总结

15. 一、迭代器：数据流的生产者

16. 二、生成💎💎💎：迭代器的优雅实现

17. 三、实战：处理大文件

18. 四、生成器的进阶用法

19. 五、性能对比与选择建议

20. 总结

编程实战项目合集

从命令行工具到 Web 应用，一步步带你完成实战项目。

目录

- 1. Docker入门：容器化你的第一个应用 (2026-05-06)
- 2. Python异步编程：asyncio实战 (2026-05-06)
- 3. Python生成器与迭代器：高效处理大数据 (2026-05-06)
- 4. Python装饰器从入门到进阶 (2026-05-06)
- 5. 用Python写一个命令行工具：argparse实战 (2026-05-06)

声明： 本内容由 AI 编程实验室自动整理生成，仅供参考学习。

=====

第 1 章：Docker入门：容器化你的第一个应用

Docker入门：容器化你的第一个应用

当你新加入一个团队，或者换了一台开发机器，最头疼的事莫过于“在我电脑上明明能跑”——环境不一致、依赖缺失、系统版本差异.....Docker 正是为了解决这个痛点而生。本文将通过一个简单的 Node.js Web 应用，带你从零开始体验容器化的完整流程。

1. 为什么需要 Docker?

传统开发中，应用运行依赖操作系统、运行时、库文件、环境变量等。哪怕只是从开发机部署到测试服务器，都可能在“环境配置”这一步翻车。

Docker 的核心思路是**将应用及其所有依赖打包成一个标准化的“容器”**，这个容器可以在任何安装了 Docker 的机器上以相同方式运行。相当于给你的应用造了一个“移动小房子”，无论搬到哪台机器，里面的环境都一模一样。

2. 准备工作：安装 Docker

在开始之前，请确保你的机器已经安装了 Docker。

macOS / Windows: 下载 [Docker Desktop](#) 并安装。Linux (Ubuntu为例)：

```
sudo apt update
sudo apt install docker.io
sudo systemctl start docker
sudo systemctl enable docker
```

安装完成后，在终端运行 `docker --version` 确认安装成功。

3. 编写一个极简的 Node.js 应用

我们在一个空目录下创建两个文件。

app.js (一个简单的 HTTP 服务)：

```
const http = require('http');
const port = 3000;

const server = http.createServer((req, res) => {
```

```
res.statusCode = 200;
res.setHeader('Content-Type', 'text/plain');
res.end('Hello, Docker!\n');
});

server.listen(port, () => {
  console.log(`Server running at http://localhost:${port}/`);
});
```

package.json (定义依赖, 这里没有额外依赖, 但需要声明入口) :

```
{
  "name": "docker-demo",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  }
}
```

现在运行 `node app.js` , 访问 `http://localhost:3000` 可以看到 "Hello, Docker!". 接下来我们把它装进 Docker 容器。

4. 编写 Dockerfile: 描述你的容器

在项目根目录创建一个名为 `Dockerfile` 的文件 (注意没有扩展名) :

```
# 使用官方 Node.js 镜像作为基础
FROM node:18-alpine

# 设置容器内的工作目录
WORKDIR /app

# 将 package.json 复制到工作目录
COPY package.json .

# 安装依赖 (这里没有依赖, 但保留此步骤)
RUN npm install

# 将🔖目所有文件复制到容器
COPY . .

# 暴露容器内的端口 3000
EXPOSE 3000
```

```
# 容器启动时执行的命令
CMD ["npm", "start"]
```

关键指令解释：

- **FROM**：指定基础镜像。**node:18-alpine** 是精简版 Node.js 镜像，体积小。
- **WORKDIR**：设置容器内的工作目录，后续命令都在此目录执行。
- **COPY**：将宿主机文件复制到容器内。
- **RUN**：在容器构建过程中执行命令（如安装依赖）。
- **EXPOSE**：声明容器运行时监听的端口（仅作文档说明，不实际映射）。
- **CMD**：容器启动时默认执行的命令。

5. 构建并运行你的第一个容器

5.1 构建镜像

在项目根目录执行：

```
docker build -t my-node-app .
```

- **-t my-node-app**：给镜像打上标签（名字），方便识别。
- **.**：指定构建上下文为当前目录。

构建成功后，运行 **docker images** 可以看到名为 **my-node-app** 的镜像。

5.2 运行容器

```
docker run -d -p 8080:3000 --name my-app my-node-app
```

- **-d**：后台运行（detached）。
- **-p 8080:3000**：将宿主机的 8080 端口映射到容器内的 3000 端口。
- **--name my-app**：给容器起个名字。

现在访问 <http://localhost:8080>，你应该能看到 "Hello, Docker!"。

5.3 常用容器操作

```
# 查看运行中的容器
docker ps

# 查看所有容器（包括停止的）
docker ps -a

# 停止容器
```

```
docker stop my-app

# 启动已停止的容器
docker start my-app

# 查看容器日志
docker logs my-app

# 删除容器（需先停止）
docker rm my-app

# 删除镜像
docker rmi my-node-app
```

6. 实际场景：为什么这样写才是最佳实践？

你可能会问：为什么要把 `package.json` 和源代码分开两次 `COPY`？这背后是 Docker 构建缓存的优化策略。

Docker 构建镜像时，每一行指令都会产生一个缓存层。当修改了源代码（`app.js`），但 `package.json` 没变时，Docker 会复用 `COPY package.json .` 和 `RUN npm install` 的缓存，只重新执行后续步骤。这能大幅缩短构建时间，尤其是在项目依赖较多时。

反例：如果写成 `COPY . .` 放在 `RUN npm install` 前面，那么每次修改任何文件都会导致整个安装依赖的步骤重新执行。

总结

通过本文，你完成了 Docker 的第一次实战：1. 理解容器化解决环境一致性问题。2. 编写了一个简单的 Node.js 应用。3. 用 `Dockerfile` 描述应用如何打包。4. 构建镜像并运行容器，验证服务正常。

接下来你可以尝试：将同一个镜像部署到另一台机器（只需安装 Docker），或者使用 `docker-compose` 编排多个容器（如 Web + 数据库）。容器化的世界，才刚刚开始。

=====

第 2 章：Python异步编程：asyncio实战

Python异步编程：asyncio实战

在开发网络爬虫、Web服务或需要处理大量I/O操作的应用时，你是否遇到过程序卡在等待响应上，导致整体性能低下？Python的 `asyncio` 库正是解决这类问题的利器。本文将带你从零掌握 `asyncio` 的核心用法，并通过实战案例提升代码执行效率。

1. 理解异步编程的核心：协程与事件循环

异步编程的核心是**协程** (Coroutine) 和**事件循环** (Event Loop)。协程是可以暂停执行并让出控制权的函数，事件循环则负责调度这些协程。

基础示例：定义和运行协程

```
import asyncio

async def greet(name):
    print(f"Hello, {name}!")
    await asyncio.sleep(1) # 模拟I/O等待
    print(f"Goodbye, {name}!")

async def main():
    # 创建协程对象，但不会立即执行
    task1 = asyncio.create_task(greet("Alice"))
    task2 = asyncio.create_task(greet("Bob"))

    # 等待所有任务完成
    await asyncio.gather(task1, task2)

# 运行事件循环
asyncio.run(main())
```

关键点： - `async def` 定义协程函数 - `await` 挂起当前协程，等待另一个协程完成 - `asyncio.run()` 是Python 3.7+推荐的启动入口

运行上述代码，你会看到两个问候语几乎同时输出，而不是顺序执行——这就是异步的魅力。

2. 实际场景：异步HTTP请求

网络请求是最常见的I/O密集型操作。我们使用 `aiohttp` 库来实现异步HTTP请求。

安装依赖

```
pip install aiohttp
```

实战：批量抓取网页标题

```
import asyncio
import aiohttp
from bs4 import BeautifulSoup

async def fetch_title(session, url):
    try:
        async with session.get(url) as response:
            html = await response.text()
            soup = BeautifulSoup(html, 'html.parser')
            title = soup.title.string if soup.title else "No title"
            print(f"{url}: {title[:50]}")
            return title
    except Exception as e:
        print(f"Error fetching {url}: {e}")
        return None

async def main():
    urls = [
        "https://www.python.org",
        "https://www.github.com",
        "https://stackoverflow.com",
        "https://www.reddit.com"
    ]

    async with aiohttp.ClientSession() as session:
        # 创建所有抓取任务
        tasks = [fetch_title(session, url) for url in urls]
        # 并发执行
        results = await asyncio.gather(*tasks)

    print(f"成功抓取 {sum(1 for r in results if r)}/{len(urls)} 个页面")

asyncio.run(main())
```

性能对比：如果使用同步方式，4个请求需要至少4秒（假设每个请求1秒）；而异步方式只需约1秒，效率提升明显。

3. 控制并发：限制同时进行的任务数

直接使用 `asyncio.gather` 会一次性发起所有任务，可能导致目标服务器过载或被封IP。我们需要限制并发数。

使用信号量控制并发

```
import asyncio
import aiohttp

async def fetch_url(semaphore, session, url):
    async with semaphore: # 获取信号量
        async with session.get(url) as response:
            return await response.text()

async def main():
    urls = [f"https://httpbin.org/delay/{i}" for i in range(1, 11)] # 模拟10个慢请求

    # 限制最多3个并发
    semaphore = asyncio.Semaphore(3)

    async with aiohttp.ClientSession() as session:
        tasks = [fetch_url(semaphore, session, url) for url in urls]
        results = await asyncio.gather(*tasks)

    print(f"完成 {len(results)} 个请求")

asyncio.run(main())
```

信号量工作原理：同时最多允许3个协程进入 `async with semaphore` 块，其他协程会排队等待。

4. 超⚡️⚡️⚡️处理与错误恢复

在实际应用中，网络请求可能超时或失败。我们需要优雅地处理这些异常。

带超时的任务管理

```
import asyncio
import aiohttp

async def fetch_with_timeout(session, url, timeout=5):
    try:
        async with asyncio.timeout(timeout): # Python 3.11+ 语法
            async with session.get(url) as response:
                return await response.text()
    except asyncio.TimeoutError:
        print(f"{url} 请求超时")
```

```

        return None
    except Exception as e:
        print(f"{url} 请求失败: {e}")
        return None

# 对于Python 3.10及以下版本, 使用asyncio.wait_for
async def fetch_with_timeout_legacy(session, url, timeout=5):
    try:
        async with session.get(url) as response:
            # 限制读取操作的时间
            return await asyncio.wait_for(response.text(), timeout=timeout)
    except asyncio.TimeoutError:
        print(f"{url} 请求超时")
        return None

```

实战建议：为每个任务设置合理的超时时间，避免单个慢请求拖垮整个程序。

5. 异步与同步代码的桥接

有时我们需要在异步代码中调用同步的第三方库。使用 `loop.run_in_executor` 可以将同步任务交给线程池执行。

混合使用示例

```

import asyncio
import time

def sync_blocking_task(n):
    """模拟一个耗时的CPU操作"""
    time.sleep(n) # 注意: 这里使用时间.sleep模拟阻塞
    return f"完成耗时{n}秒的任务"

async def main():
    loop = asyncio.get_running_loop()

    # 将同步任务提交到默认线程池
    result = await loop.run_in_executor(None, sync_blocking_task, 2)
    print(result)

    # 也可以指定线程池
    from concurrent.futures import ThreadPoolExecutor
    with ThreadPoolExecutor(max_workers=4) as pool:
        results = await asyncio.gather(*[
            loop.run_in_executor(pool, sync_blocking_task, i)
            for i in range(1, 4)
        ])

```

```
print(results)

asyncio.run(main())
```

注意： `run_in_executor` 适用于CPU密集型或阻塞的同步代码，但不要滥用——异步的核心优势是处理I/O等待。

总结

本文带你走过了asyncio实战的四个关键环节：1. **协程与事件循环**：理解 `async/await` 语法和 `asyncio.run` 入口 2. **异步HTTP请求**：使用 `aiohttp` 实现并发网络请求 3. **并发控制**：用 `Semaphore` 限制同时执行的任务数 4. **错误处理**：添加超时和异常处理机制

记住，异步编程不是银弹——它最适合I/O密集型任务（网络请求、文件读写、数据库操作），对于CPU密集型计算（图像处理、加密解密），请考虑使用多进程或多线程。现在，打开你的编辑器，将学到的技术应用到下一个项目中吧！

=====

第 3 章：Python生成器与迭代器：高效处理大数据

Python生成器与迭代器：高效处理大数据

当你的Python程序需要处理数百万行日志、超大CSV文件或无限数据流时，直接使用列表存储所有数据往往会耗尽内存，甚至导致程序崩溃。生成器与迭代器正是解决这类问题的利器——它们通过**惰性计算**方式，只在需要时生成数据，从而大幅降低内存占用。

一、迭代器：数据流的生产者

任何实现了 `__iter__()` 和 `__next__()` 方法的对象都是迭代器。它像一个“数据管道”，每次调用 `next()` 才吐出下一个元素。

基础示例：手动实现迭代器

```
class Counter:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= self.end:
            raise StopIteration
        value = self.current
        self.current += 1
        return value

# 使用迭代器
counter = Counter(1, 5)
for num in counter:
    print(num) # 输出: 1 2 3 4
```

关键点： `for` 循环本质上是不断调用 `next()` 直到捕获 `StopIteration` 异常。

二、生成器🔗🔗🔗：迭代器的优雅实现

生成器是Python提供了一种更简洁的迭代器创建方式。用 `yield` 关键字替代 `return`，函数就变成了生成器函数。

对比：列表 vs 生成器

```
# 列表方式（一次性生成所有数据）
def get_squares_list(n):
    return [x**2 for x in range(n)]

# 生成器方式（按需生成）
def get_squares_gen(n):
    for x in range(n):
        yield x**2

# 测试内存占用
import sys
squares_list = get_squares_list(100000)
squares_gen = get_squares_gen(100000)

print(f"列表大小: {sys.getsizeof(squares_list)} bytes") # 约824456
print(f"生成器大小: {sys.getsizeof(squares_gen)} bytes") # 约112
```

实际效果：生成器对象无论生成多少数据，自身大小恒定，而列表会随着数据量线性增长。

生成器表达式：一行代码搞定

```
# 列表推导式
list_comp = [x*2 for x in range(1000)]

# 生成器表达式（只需将方括号改为圆括号）
gen_expr = (x*2 for x in range(1000))

print(type(gen_expr)) # <class 'generator'>
```

三、实战：处理大文件

假设有一个10GB的日志文件 `access.log`，我们需要统计每个IP的访问次数。

错误做法（内存爆炸）

```
def parse_log_bad(filepath):
    with open(filepath) as f:
        lines = f.readlines() # 一次性加载全部内容到内存
        ip_counts = {}
        for line in lines:
            ip = line.split()[0]
```

```
ip_counts[ip] = ip_counts.get(ip, 0) + 1
return ip_counts
```

正确做法（流式处理）

```
def parse_log_good(filepath):
    ip_counts = {}
    with open(filepath) as f:
        for line in f: # 文件对象本身就是迭代器，逐行读取
            ip = line.split()[0]
            ip_counts[ip] = ip_counts.get(ip, 0) + 1
    return ip_counts

# 或者使用生成器函数封装读取逻辑
def read_lines(filepath):
    with open(filepath) as f:
        for line in f:
            yield line.strip()

# 使用生成器处理
for line in read_lines("huge_log.txt"):
    process(line) # 逐行处理，内存占用极低
```

原理：`open()` 返回的文件对象是一个迭代器，`for line in f` 不会将整个文件读入内存，而是每次读取一行。

四、生成器的进阶用法

1. 无限序列生成

```
def fibonacci():
    a, b = 0, 1
    while True: # 无限循环
        yield a
        a, b = b, a + b

# 只取前10个
fib = fibonacci()
first_10 = [next(fib) for _ in range(10)]
print(first_10) # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

2. 生产器管道（数据流水线）

```
# 构建处理管道
def read_data(filepath):
    with open(filepath) as f:
        for line in f:
            yield line.strip()

def filter_empty(data):
    for item in data:
        if item: # 过滤空行
            yield item

def to_uppercase(data):
    for item in data:
        yield item.upper()

# 串联管道
pipeline = to_uppercase(filter_empty(read_data("data.txt")))
for processed_line in pipeline:
    print(processed_line)
```

3. 使用 `yield from` 简化嵌套生成器

```
def chain(*iterables):
    for it in iterables:
        yield from it # 委托给子迭代器

# 合并多个列表
combined = chain([1,2,3], "abc", [4,5])
print(list(combined)) # [1, 2, 3, 'a', 'b', 'c', 4, 5]
```

五、性能对比与选择建议

场景	推荐方式	原因
数据量 < 1000条	列表	简单直观，访问速度快
数据量 > 10000条	生成器/迭代器	内存占用低，避免OOM
需要多次遍历	列表/重新创建生成器	生成器只能遍历一次
无限数据流	生成器	无法用列表表示无限序列

测试代码：比较列表和生成器的性能

```
import time

# 列表方式
start = time.time()
result = sum([x for x in range(10_000_000)])
print(f"列表耗时: {time.time() - start:.2f}s")

# 生成器方式
start = time.time()
result = sum(x for x in range(10_000_000))
print(f"生成器耗时: {time.time() - start:.2f}s")
# 两者耗时相近，但生成器内存占用几乎为0
```

总结

- **迭代器**是实现了 `__iter__` 和 `__next__` 的对象，支持惰性计算
- **生成器**是用 `yield` 创建的迭代器，更简洁易用
- 处理大文件、无限序列时，优先使用生成器
- 生成器表达式是列表推导式的内存友好版本
- 通过 `yield from` 可以轻松构建数据管道

掌握生成器和迭代器，你的Python程序就能以极低的内存开销处理海量数据——这正是Python在数据处理领域广受欢迎的重要原因之一。

=====

第 4 章：Python装饰器从入门到进阶

Python装饰器从入门到进阶

装饰器是Python中一个强大而优雅的特性，它允许你在不修改原函数代码的情况下，动态地给函数添加新的功能。本文将带你从基础语法到高级应用，彻底掌握装饰器的使用。

一、理解装饰器的核心概念

装饰器本质上是一个接收函数作为参数并返回新函数的高阶函数。让我们从最简单的例子开始：

```
def my_decorator(func):
    def wrapper():
        print("在函数执行前做一些事情")
        func()
        print("在函数执行后做一些事情")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

输出：

```
在函数执行前做一些事情
Hello!
在函数执行后做一些事情
```

`@my_decorator` 语法糖等价于 `say_hello = my_decorator(say_hello)`。 `wrapper` 函数包裹了原函数，在调用前后添加了额外逻辑。

二、处理带参数的函数

上面的装饰器只能处理无参函数。为了让装饰器通用，我们需要使用 `*args` 和 `**kwargs`：

```
def timer_decorator(func):
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
```

```

        result = func(*args, **kwargs)
    end = time.time()
    print(f"{func.__name__} 执行耗时: {end - start:.4f}秒")
    return result
return wrapper

@timer_decorator
def calculate_sum(n):
    return sum(range(n))

@timer_decorator
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

result = calculate_sum(1000000)
print(f"计算结果: {result}")
greet("Alice", "Hi")

```

这个装饰器可以测量任意函数的执行时间，并保持原函数的返回值。

三、保留原函数的元信息

使用 `functools.wraps` 可以保留被装饰函数的名称、文档字符串等元信息：

```

from functools import wraps

def log_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"调用函数: {func.__name__}")
        print(f"参数: {args}, {kwargs}")
        result = func(*args, **kwargs)
        print(f"返回值: {result}")
        return result
    return wrapper

@log_decorator
def add(a, b):
    """计算两个数的和"""
    return a + b

print(f"函数名: {add.__name__}")
print(f"文档: {add.__doc__}")
add(3, 5)

```

没有 `@wraps` 时，`add.__name__` 会变成 `wrapper`，这会导致调试困难。

四、带参数的装饰器

有时我们需要给装饰器传递参数，比如设置日志级别。这需要再嵌套一层函数：

```
def repeat(times):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(times=3)
def say_hi():
    print("Hi!")

say_hi()
```

这里的 `repeat(times=3)` 先用返回 `decorator` 函数，然后 `@decorator` 再应用于 `say_hi`。

五、实际应用场景

1. 权限验证

```
def require_auth(role):
    def decorator(func):
        @wraps(func)
        def wrapper(user, *args, **kwargs):
            if user.get("role") != role:
                raise PermissionError(f"需要{role}权限")
            return func(user, *args, **kwargs)
        return wrapper
    return decorator

@require_auth("admin")
def delete_user(user, user_id):
    print(f"用户 {user_id} 已被删除")

# 测试
admin_user = {"name": "Admin", "role": "admin"}
normal_user = {"name": "User", "role": "user"}
```



```
delete_user(admin_user, 123) # 正常执行
# delete_user(normal_user, 123) # 抛出PermissionError
```

2. 缓存计算结果

```
def cache_result(func):
    cache = {}
    @wraps(func)
    def wrapper(*args):
        if args in cache:
            print(f"使用缓存: {args}")
            return cache[args]
        result = func(*args)
        cache[args] = result
        return result
    return wrapper

@cache_result
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(10)) # 第一次计算
print(fibonacci(10)) # 直接从缓存获取
```

3. 重试机制

```
import time

def retry(max_attempts=3, delay=1):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts - 1:
                        raise
                    print(f"第{attempt + 1}次失败, {delay}秒后重试...")
                    time.sleep(delay)
            return wrapper
        return decorator

    @retry(max_attempts=3, delay=0.5)
    def unstable_network_call():
```

```
import random
if random.random() < 0.7:
    raise ConnectionError("网络异常")
return "成功获取数据"

print(unstable_network_call())
```

六、多个装饰器的组合

当多个装饰器叠加时，执行顺序❖❖❖从下往上（离函数最近的先执行）：

```
@timer_decorator
@log_decorator
def complex_function(x, y):
    return x ** y

complex_function(2, 10)
```

这相当于 `complex_function = timer_decorator(log_decorator(complex_function))`，先执行 `log_decorator`，再执行 `timer_decorator`。

总结

装饰器是Python中实现AOP（面向切面编程）的优雅方式。核心要点：

1. **基本结构**：装饰器是接收函数返回函数的高阶函数
2. **参数传递**：使用 `*args`, `**kwargs` 保持通用性
3. **元信息保留**：始终使用 `@wraps` 装饰wrapper函数
4. **带参数装饰器**：需要三层嵌套函数
5. **实际应用**：日志、权限、缓存、重试等场景非常实用

掌握装饰器后，你会发现代码复用性和可维护性都得到了显著提升。

=====

第 5 章：用Python写一个命令行工具：argparse实战

用Python写一个命令行工具：argparse实战

你是否写过这样的脚本：每次运行都要手动修改代码里的文件路径、参数数值？或者想给同事提供一个易用的工具，却不知道如何处理命令行参数？Python标准库中的 `argparse` 模块正是解决这些问题的利器。本文将通过三个实战场景，带你掌握argparse的核心用法。

基础入门：从零搭建参数解析框架

首先，我们用最简单的例子认识argparse的工作流程。假设我们要写一个文件处理工具，需要接收文件名和操作模式。

```
import argparse

def main():
    parser = argparse.ArgumentParser(description='文件处理工具')
    parser.add_argument('filename', help='要处理的文件名')
    parser.add_argument('--mode', choices=['read', 'write'], default='read',
                        help='操作模式: read或write')

    args = parser.parse_args()
    print(f'处理文件: {args.filename}')
    print(f'操作模式: {args.mode}')

if __name__ == '__main__':
    main()
```

运行效果：

```
$ python file_tool.py data.txt --mode write
处理文件: data.txt
操作模式: write
```

关键点解析：

- `add_argument()` 的第一个参数是参数名，不带 `--` 的是位置参数（必须提供），带 `--` 的是可选参数
- `choices` 限制参数值范围，`default` 设置默认值
- `help` 为每个参数添加说明，运行 `python file_tool.py -h` 时会自动生成帮助信息

进阶实战：构建多功能计算器

现在我们来做一个更实用的例子：一个支持加、减、乘、除的计算器，可以处理任意数量的数字，还能选择输出格式。

```
import argparse

def calculate(numbers, operation):
    if operation == 'add':
        return sum(numbers)
    elif operation == 'sub':
        result = numbers[0]
        for n in numbers[1:]:
            result -= n
        return result
    elif operation == 'mul':
        result = 1
        for n in numbers:
            result *= n
        return result
    elif operation == 'div':
        result = numbers[0]
        for n in numbers[1:]:
            result /= n
        return result

def main():
    parser = argparse.ArgumentParser(description='多功能计算器')
    parser.add_argument('numbers', nargs='+', type=float,
                        help='要计算的数字，至少两个')
    parser.add_argument('-o', '--operation', choices=['add', 'sub', 'mul', 'div'],
                        default='add', help='运算类型')
    parser.add_argument('--precision', type=int, default=2,
                        help='结果保留小数位数')
    parser.add_argument('-v', '--verbose', action='store_true',
                        help='显示详细计算过程')

    args = parser.parse_args()

    if len(args.numbers) < 2:
        parser.error('至少需要两个数字')

    result = calculate(args.numbers, args.operation)

    if args.verbose:
        op_symbols = {'add': '+', 'sub': '-', 'mul': '*', 'div': '/'}
        expression = f' {op_symbols[args.operation]} '.join(
            [str(n) for n in args.numbers])
        print(f'计算表达式: {expression}')
        print(f'运算类型: {args.operation}')
```

```
print(f'结果: {result:.{args.precision}f}')

if __name__ == '__main__':
    main()
```

运行示例:

```
$ python calculator.py 10 5 3 -o sub --precision 3 -v
计算表达式: 10 - 5 - 3
运算类型: sub
结果: 2.000
```

新技能点: - `nargs=+` 表示接受一个或多个参数, `type=float` 自动转换类型 -

`action='store_true'` 创建布尔开关, 指定 `-v` 时值为 True - `--precision` 展示如何定义整数参数

实战技巧: 子命令与配置文件

当你的工具功💎逐渐增多时, 可以使用子命令来组织。下面是一个数据处理工具, 包含 `convert` 和 `analyze` 两个子命令。

```
import argparse
import json

def handle_convert(args):
    print(f'转换文件: {args.input} -> {args.output}')
    print(f'编码: {args.encoding}')
    if args.pretty:
        print('启用美化输出')

def handle_analyze(args):
    print(f'分析文件: {args.input}')
    print(f'统计指标: {args.metrics}')
    if args.quiet:
        print('静默模式')

def main():
    parser = argparse.ArgumentParser(description='数据处理工具')
    parser.add_argument('--config', help='配置文件路径')

    subparsers = parser.add_subparsers(dest='command', help='可用命令')

    # convert 子命令
    convert_parser = subparsers.add_parser('convert', help='文件格式转换')
    convert_parser.add_argument('input', help='输入文件')
```

```

convert_parser.add_argument('output', help='输出文件')
convert_parser.add_argument('--encoding', default='utf-8', help='文件编码')
convert_parser.add_argument('--pretty', action='store_true', help='美化输出')

# analyze 子命令
analyze_parser = subparsers.add_parser('analyze', help='数据分析')
analyze_parser.add_argument('input', help='输入文件')
analyze_parser.add_argument('--metrics', nargs='+',
                             choices=['count', 'mean', 'max', 'min'],
                             default=['count'], help='统计指标')
analyze_parser.add_argument('--quiet', action='store_true', help='减少输出')

args = parser.parse_args()

# 支持从配置文件读取参数
if args.config:
    with open(args.config, 'r') as f:
        config = json.load(f)
        for key, value in config.items():
            if hasattr(args, key) and getattr(args, key) is None:
                setattr(args, key, value)

if args.command == 'convert':
    handle_convert(args)
elif args.command == 'analyze':
    handle_analyze(args)
else:
    parser.print_help()

if __name__ == '__main__':
    main()

```

运行效果:

```

$ python data_tool.py convert input.csv output.json --pretty
转换文件: input.csv -> output.json
编码: utf-8
启用美化输出

$ python data_tool.py analyze data.csv --metrics mean max
分析文件: data.csv
统计指标: ['mean', 'max']

```

高级特性: - `add_subparsers()` 创建子命令体系, `dest='command'` 用于判断用户输入了哪个子命令
 - 支持从JSON配置文件加载参数, 方便批量使用 - 不同子命令可以有完全不同的参数集合

总结

通过三个实战案例，我们掌握了argparse的核心用法： 1. **基础用法**：位置参数、可选参数、类型转换、参数验证 2. **进阶技巧**：多值参数、布尔开关、自定义错误提示 3. **高级应用**：子命令体系、配置文件支持

argparse的强大之处在于：它自动生成帮助信息、处理错误输入、支持类型转换，让开发者可以专注于业务逻辑。下次当你需要给脚本添加命令行参数时，不妨试试argparse，让你的工具更加专业易用。

关于本产品

- 来源：AI 编程实验室
- 文章数：5 篇
- 生成时间：2026-05-06T05:47:06.492Z

如果觉得有帮助，欢迎分享给需要的朋友！

— END —

由 AI 编程实验室自动生成

<https://hugo-xu844.github.io/autopilot-content/>